

A joining thread

Changes from previous version

- This paper revises P0206R0.
- Removed the motivation elaboration, it can be found in the previous revision.
- Removed the discarded solutions.
- Added explanation for the `std::thread`-accessing functions and discarded alternatives for it.
- Renamed `safe_thread` to `joining_thread`.
- Used `thread::id` and `thread::native_handle_type` in `joining_thread`.
- Updated proposed wording.

Abstract

C++ continues not to provide a thread type that would `join()` automatically on scope exit. This causes exception-safety problems, because failing to `join()` in all code paths causes the destructor of a `std::thread` to `terminate()`. This paper provides a solution that adds a new thread type that joins in its destructor, and is based on large-group and LEWG feedback given in Jacksonville. The proposal has been implemented and tested.

Contents

- [Solution 2: Add a new thread type that auto-joins](#)
- [What LEWG recommended in Jacksonville](#)
- [The `std::thread`-accessing function](#)
- [Wording for Solution 2](#)

Solution 2: Add a new thread type that auto-joins

In a large-group discussion in Jacksonville, the outcome was that a new thread type that joins in its destructor should be added. The benefit of this solution is that it's non-intrusive; all existing users of `std::thread` are unaffected, for better or worse. Replacing the uses of `std::thread` with the uses of the new type where need be is arguably straightforward.

What LEWG recommended in Jacksonville

LEWG guidance from Jacksonville was as follows:

1. The name of the type should not be `"safe_thread"`, but LEWG had no consensus on which name to pick. There was no consensus between `"joining_thread"` and `"simple_thread"`.
2. The thread-accessing function that converts the new thread type to `thread` should not be a conversion operator, but rather a named function, with preference, but no consensus, for `"as_thread"` and `"get_thread"`

This paper provides a type named `"joining_thread"`, with the thread conversion function `"as_thread"`. The name `"joining_thread"` describes the functionality, and doesn't suggest that it's significantly simpler. While the author feels sympathy for it perhaps being, in many cases, simpler to use, the choice is based on more accurately describing what the type does. The author doesn't think that suggests that `std::thread` doesn't join, but since `std::thread` doesn't do a join in its destructor, the chosen name seems palatable. The choice of `"as_thread"` rather than `"get_thread"` is fairly arbitrary.

The `std::thread`-accessing function

In addition to picking the name `"as_thread"`, this paper provides two `std::thread`-accessing functions:

```
thread& as_thread(); // addition to std::thread interface
const thread& as_thread() const; // addition to std::thread interface
```

The functions are intentionally not ref-qualified, they intentionally return a reference rather than a value, and it's intentional that there are no overloads that return rvalue references.

The lack of ref-qualified overloads is because of simplicity. Rather than having to write

```
joining_thread make_joining_thread();
joining_thread jt = make_joining_thread();
thread t = std::move(jt).as_thread();
```

we can write

```
joining_thread make_joining_thread();
thread t = std::move(make_joining_thread()).as_thread();
```

However, we do not want to allow

```
joining_thread make_joining_thread();
thread t{make_joining_thread()};
```

because that makes the conversion subtle. We also do not want to allow

```
joining_thread make_joining_thread();
thread t{make_joining_thread().as_thread()};
```

because that requires an overload that returns an rvalue reference (complicating the type) and makes the ownership transfer subtle.

The lack of a direct way to return an rvalue reference from an `joining_thread` rvalue does disturb forwarding cases to some extent. That was deemed acceptable, because forwarding was seen as yet another case where ownership transfer is subtle.

Finally, these are named functions rather than conversion functions. While conversion functions are more generic, it was deemed desirable that the conversion from a joining thread to a non-joining thread doesn't happen subtly in generic code.

Wording for Solution 2

In [thread.threads]/1, insert as follows:

```
Header <thread> synopsis
namespace std {
    class thread;
    class joining_thread;

    void swap(thread& x, thread& y) noexcept;
    void swap(joining_thread& x, joining_thread& y) noexcept;
```

In [thread.thread.constr]/4, insert as follows:

Remarks: This constructor shall not participate in overload resolution if `decay_t<F>` is the same type as `std::thread` or if `decay_t<F>` is the same type as `std::joining_thread`.

After [thread.thread.this], add a new section as follows:

```
Class joining_thread [thread.joining_thread.class]

namespace std {
    class joining_thread {
    public:
        // types
        typedef thread::native_handle_type native_handle_type;
        typedef thread::id id;
        // construct/copy/destroy:
        joining_thread() noexcept;
        template <class F, class ...Args> explicit joining_thread(F&& f, Args&&... args);
        ~joining_thread() noexcept; // semantics different from std::thread
        joining_thread(const joining_thread&) = delete;
        joining_thread(joining_thread&) noexcept;
        explicit joining_thread(thread&& x) noexcept; // addition to std::thread interface
```

```

    joining_thread& operator=(const joining_thread&) = delete;
    joining_thread& operator=(joining_thread&&) noexcept; // semantics different from std::thread
    joining_thread& operator=(thread&& x) noexcept; // addition to std::thread interface
    // members:
    void swap(joining_thread&) noexcept;
    bool joinable() const noexcept;
    void join();
    void detach();
    thread::id get_id() const noexcept;
    thread::native_handle_type native_handle(); // See 30.2.3
    thread& as_thread(); // addition to std::thread interface
    const thread& as_thread() const; // addition to std::thread interface
    // static members:
    static unsigned hardware_concurrency() noexcept;
};
}

```

The class `joining_thread` provides the same facilities as `thread`, and has the same members and the same semantics, with the differences as described below.

`joining_thread` constructors [thread.joining_thread.constr]

```
explicit joining_thread(thread&& x) noexcept;
```

Effects: Constructs an object of type `joining_thread` from `x`, and sets `x` to a default constructed state.

Postconditions: `x.get_id() == id()` and `get_id()` returns the value of `x.get_id()` prior to the start of construction.

`joining_thread` destructor [thread.joining_thread.destr]

```
~joining_thread() noexcept;
```

Effects: If `joinable()`, calls `join()`. Otherwise, has no effects.
 [Note: Because `~joining_thread` is required to be `noexcept`, if `join()` throws then `std::terminate()` will be called.
 --end note]

`joining_thread` assignment [thread.joining_thread.assign]

```
joining_thread& operator=(joining_thread&& x) noexcept;
```

Effects: If `joinable()`, calls `join()`. Then, assigns the state of `x` to `*this` and sets `x` to a default constructed state.
 [Note: If `join()` throws then `std::terminate()` will be called.
 --end note]
 Postconditions: `x.get_id() == id()` and `get_id()` returns the value of `x.get_id()` prior to the assignment.

Returns: `*this`

```
joining_thread& operator=(thread&& x) noexcept;
```

Effects: If `joinable()`, calls `join()`. Then, assigns the state of `x` to `*this` and sets `x` to a default constructed state.
 [Note: If `join()` throws then `std::terminate()` will be called.
 --end note]
 Postconditions: `x.get_id() == id()` and `get_id()` returns the value of `x.get_id()` prior to the assignment.

Returns: `*this`

`joining_thread` members [thread.joining_thread.member]

```
thread& as_thread();
```

Returns: A reference to the underlying thread object.

```
const thread& as_thread() const;
```

Returns: A reference to the underlying thread object.